

Guía de Laboratorio: Estructura de Repetición *for*

Resolución de Problemas, Método de Polya y Código Python | TUASySL - CURZAS UNCo

Indicaciones Metodológicas y Reglas de Cátedra

Objetivo de la Actividad: Diseñar e implementar algoritmos utilizando la estructura de repetición fija *for* en Python, aplicando el **Método de Polya** para la abstracción de información superflua y resolviendo problemas en 20 minutos.

Pautas de Estilo Cátedra (PEP 8): Utilizar nomenclatura *snake_case* para variables/funciones (ej: *total_exceso_kwh*) y *SCREAMING_SNAKE_CASE* para constantes (ej: *UMBRAL_EFICIENCIA*).

Estructura de Análisis: Plantilla del Método de Polya

Fase del Método	Acciones Técnicas a Desarrollar
1. Comprender el problema	Identificar la incógnita, datos de entrada, constantes y <i>filtrar la información superflua de contexto (ruido)</i> .
2. Concebir un plan	Diseñar la estrategia iterativa identificando el rango/secuencia del <i>for</i> , acumuladores y condicionales.
3. Ejecutar el plan	Escribir el script limpio en Python siguiendo las normas PEP 8 de la cátedra.
4. Visión retrospectiva	Realizar la prueba de escritorio / traza de memoria manual para verificar las salidas obtenidas.

Problema 1: Control Fijo de Consumo de Energía en Servidores

Enunciado con Ruido de Contexto: El centro de cómputos del edificio central del CURZAS en Viedma posee un panel de monitoreo conectado a una red de fibra óptica de 1 Gbps. Durante las jornadas de verano, la temperatura de la sala alcanza los 28 °C y el consumo de los racks se mide en kWh. **Consigna:** Escribir un script en Python que, utilizando un ciclo *for* con *range(1, 6)*, solicite al operador ingresar el consumo en kWh (float) de 5 servidores. Usar la constante *UMBRAL_EFICIENCIA* = 120.0. Si el consumo supera dicho umbral, sumar el exceso (consumo - *UMBRAL_EFICIENCIA*) al acumulador *total_exceso_kwh*. Al finalizar, imprimir el total acumulado.

```
# Constante global de la cátedra
UMBRAL_EFICIENCIA = 120.0

# Variable acumuladora
total_exceso_kwh = 0.0

# Ciclo for para registrar secuencialmente los 5 servidores
for servidor in range(1, 6):
    consumo_actual = float(input(f'Ingrese consumo kWh servidor {servidor}: '))
    if consumo_actual > UMBRAL_EFICIENCIA:
        exceso = consumo_actual - UMBRAL_EFICIENCIA
        total_exceso_kwh += exceso

print(f'Total acumulado de consumo en exceso: {total_exceso_kwh} kWh')
```

Problema 2: Auditoría de Claves Institucionales

Enunciado con Ruido de Contexto: La Tecnicatura Universitaria en Administración de Sistemas y Software Libre (TUASySL) dicta sus clases teóricas en el aula 3 equipada con proyector HD de 3500 lúmenes. Para acceder al campus virtual de la Universidad Nacional del Comahue, los 45 estudiantes inscriptos deben utilizar una contraseña segura que cumpla con los estándares institucionales. **Consigna:** Escribir un script que pida ingresar la clave de acceso (str). Recorrer la cadena caracter por caracter con un bucle for contando las apariciones del símbolo '#'. Definir la constante MINIMO_CARACTERES_ESPECIALES = 2. Mediante if-else, si el conteo es >= constante, imprimir 'Clave Segura Aprobada', de lo contrario imprimir 'Clave Inválida: Se requieren al menos 2 símbolos #'.

```
# Constante global
MINIMO_CARACTERES_ESPECIALES = 2
clave_usuario = input('Ingrese su clave institucional: ')
cantidad_simbolos = 0
# Iteración caracter por caracter sobre la cadena de texto
for caracter in clave_usuario:
    if caracter == '#':
        cantidad_simbolos += 1
if cantidad_simbolos >= MINIMO_CARACTERES_ESPECIALES:
    print('Clave Segura Aprobada')
else:
    print('Clave Inválida: Se requieren al menos 2 símbolos #')
```

Problema 3: Control de Asistencia Semanal a Laboratorio

Enunciado con Ruido de Contexto: La biblioteca central del CURZAS en la orilla del río Negro atiende en el horario de 8:00 a 20:00 hs y posee un catálogo físico de más de 4.500 libros. Para las prácticas de laboratorio en computadoras, los estudiantes deben registrar su presencia mediante un código diario durante los 5 días hábiles de la semana. **Consigna:** Escribir un script que solicite una cadena de 5 caracteres (str) representando la asistencia semanal (ej: 'P-P-A-P-P'). Recorrer la cadena con for contando las 'P'. Definir constantes MINIMO_PROMOCION = 4 y MINIMO_REGULAR = 3. Evaluar con if-elif-else e imprimir si está Promocionado, Regular o Libre.

```
MINIMO_PROMOCION = 4
MINIMO_REGULAR = 3
registro_semana = input('Ingrese el registro semanal de 5 días (P/A): ')
cant_presentes = 0
for estado in registro_semana:
    if estado == 'P':
        cant_presentes += 1
if cant_presentes >= MINIMO_PROMOCION:
    print('Condición: Promocionado en Asistencia')
elif cant_presentes >= MINIMO_REGULAR:
    print('Condición: Regular en Asistencia')
else:
    print('Condición: Libre por Inasistencias')
```

Problema 4: Promedio de Parciales Módulos Aprobados

Enunciado con Ruido de Contexto: El equipo docente de Introducción a la Programación publica los resultados de los parciales en la plataforma PEDCO / Moodle. La materia contempla un régimen de evaluación continua con 8 parciales cortos a lo largo del cuatrimestre, tomados los días martes en el laboratorio informático. **Consigna:** Escribir un programa que utilice `range(1, 9)` en un ciclo `for` para solicitar las notas de los 8 parciales (`float`). Usar la constante `NOTA_APROBACION = 6.0`. Si la nota es `>=` constante, acumular el puntaje e incrementar el contador. Al finalizar, si hay aprobados, calcular e imprimir el promedio; de lo contrario, informar que no hubo aprobados.

```
NOTA_APROBACION = 6.0
suma_notas_aprobadas = 0.0
cant_aprobados = 0
for parcial in range(1, 9):
    nota_ingresada = float(input(f'Ingrese la nota del parcial {parcial}: '))
    if nota_ingresada >= NOTA_APROBACION:
        suma_notas_aprobadas += nota_ingresada
        cant_aprobados += 1
if cant_aprobados > 0:
    promedio_aprobados = suma_notas_aprobadas / cant_aprobados
    print(f'Promedio de parciales aprobados: {promedio_aprobados:.2f}')
else:
    print('No se registraron parciales aprobados')
```